# CPU Scheduling

## DSC 315: Computer Organization & Operating Systems

**Dr. Laltu Sardar**

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)

March 25 & 27, 2026

# Table of Content

1. CPU Scheduling
   - Basic Concepts
   - Scheduling Algorithms
   - Multi-Processor Scheduling

**Mandatory** Reading From textbook:
Chapters 5.1, 5.2, 5.3, 5.5

5.1

# Basic Concepts

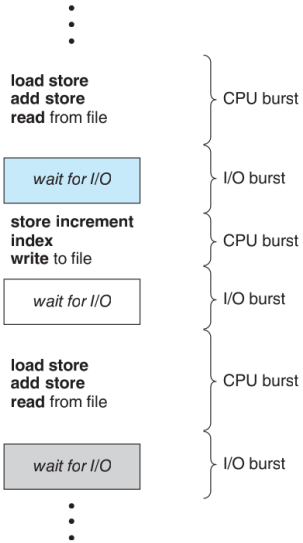
# Basic Concepts

## CPU Scheduling

- Scheduling is used for efficient utilization of CPU.
- CPU is one of the primary resources in a system.

## CPU–I/O Burst Cycle

- Process execution alternates between:
  - CPU burst
  - I/O burst
- Process starts with CPU burst and ends with CPU burst.
- This cycle continues throughout process lifetime.



# Schedulers and Dispatcher

**Schedulers**

- After every I/O burst, process enters ready queue.
- Scheduler selects the next process to execute.
- Queues used:
  - FIFO queue
  - Priority queue

**Dispatcher**

- Transfers CPU control to selected process.

**Context Switching**

- Switching CPU from one process to another.
- Requires saving and restoring process state.
- Overhead depends on hardware.
- Measured in milliseconds.

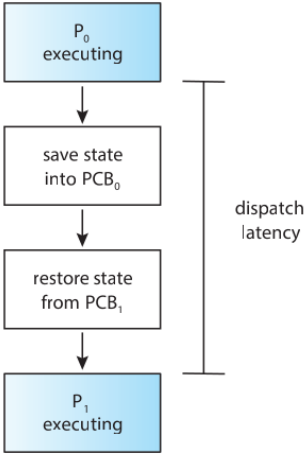



Figure: Dispatcher and Context Switching

# Scheduling Decision (5.2)

**When Scheduling Occurs**

1. Running → Waiting
2. Running → Ready
3. Waiting → Ready
4. Process terminates

**Types of Scheduling**

- Non-preemptive: (1) and (4)
- Preemptive: all cases

**Modern Systems**

- Mostly preemptive

# Scheduling Criteria

1. **CPU Utilization**
   - Keeps CPU busy
2. **Throughput**
   - Number of processes completed per unit time
3. **Turnaround Time**
   - Submission to completion time
4. **Waiting Time**
   - Time spent in ready queue
5. **Response Time**
   - Time from submission to first response

5.3

# CPU Scheduling Algorithms

# First Come First Serve (FCFS) (5.3.1)

**Assumptions**

- Single CPU
- One process executes at a time

**Example**

- P1 = 24, P2 = 3, P3 = 3

**Execution Order**

- P1 $\rightarrow$ P2 $\rightarrow$ P3

# FCFS Waiting Time Analysis

**Average Waiting Time**

$$\frac{0 + 24 + 27}{3} = 17$$

**Improved Order**

- P2 $\rightarrow$ P3 $\rightarrow$ P1

$$\frac{0 + 3 + 6}{3} = 3$$

**Observation**

- Order significantly affects performance

# FCFS Drawbacks

- Convoy effect:
  - Short processes wait behind long processes
- Non-preemptive
- Poor for interactive systems

# Shortest Job First (SJF) (5.3.2)

**Idea**

- Select process with shortest CPU burst

**Problem**

- Burst time is not known in advance

**Example**

- P1 = 6, P2 = 8, P3 = 7, P4 = 3

**Execution**

- P4 $\rightarrow$ P1 $\rightarrow$ P3 $\rightarrow$ P2

# SJF Properties

**Average Waiting Time**

$$\frac{0+3+9+16}{4} = 7$$

**Key Property**

- SJF gives minimum average waiting time

# Burst Time Prediction

**Exponential Averaging**

$$T_{n+1} = \alpha t_n + (1 - \alpha) T_n$$

**Special Case**

$$T_{n+1} = \frac{t_n + T_n}{2} \quad (\alpha = 1/2)$$

**Interpretation**

- Uses past values to predict next CPU burst

# Shortest Remaining Time First (SRTF)

**Definition**

- Preemptive version of SJF
- Process with smallest remaining time executes

**Example**

- Arrival: P1=0, P2=1, P3=2, P4=3
- Burst: P1=8, P2=4, P3=9, P4=5

# Round Robin (RR) (5.3.3)

**Concept**

- Each process gets fixed time quantum
- After quantum expires:
  - Process goes to end of queue

**Characteristics**

- Small quantum $\rightarrow$ many context switches
- Large quantum $\rightarrow$ behaves like FCFS

**Rule of Thumb**

- Quantum $\approx$ 80% of CPU bursts

# Priority Scheduling

**Definition**

- Highest priority process executes first

**Types**

- Internal priority
- External priority

**Issue: Starvation**

- Low priority processes may never execute

**Solution: Aging**

- Increase priority over time

# Multilevel Queue Scheduling

- Multiple queues based on process type
- Each queue has its own scheduling algorithm
- Scheduling between queues required

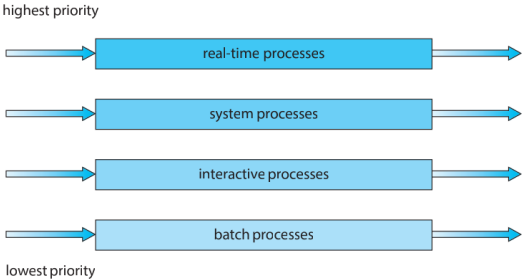



Figure: Multilevel queue scheduling.

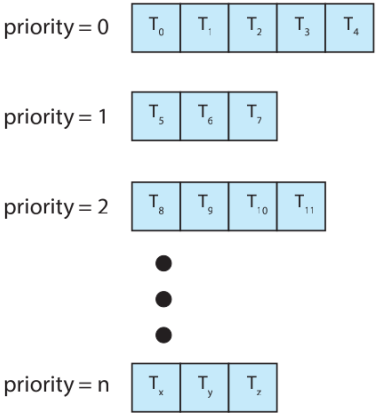



Figure: Separate queues for each priority.

# Multilevel Feedback Queue

- A process can move between the various queues.

**Multilevel-feedback-queue scheduler is defined by:**

- Number of queues
- Scheduling algorithm for each queue
- Method to determine when to upgrade a process
- Method to determine when to demote a process
- Method to determine which queue a process will enter when it needs service

**Note:**

- Aging can be implemented using multilevel feedback queue

# Example of Multilevel Feedback Queue

**Three queues:**

- Q0 – RR with time quantum 8 milliseconds
- Q1 – RR with time quantum 16 milliseconds
- Q2 – FCFS

**Scheduling**

- A new process enters queue Q0 which is served in RR
  - When it gains CPU, the process receives 8 milliseconds
  - If it does not finish in 8 milliseconds, it is moved to Q1
- At Q1 job is again served in RR and receives 16 additional milliseconds
  - If it still does not complete, it is preempted and moved to Q2

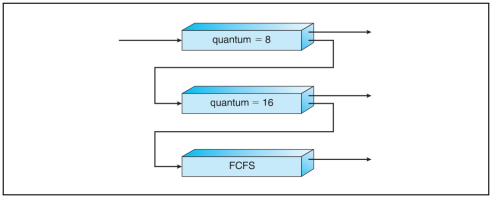



Figure: Multilevel Feedback Queue Example

# List of Scheduling Algorithms

- CPU scheduling improves utilization
- Multiple algorithms exist:
  - FCFS
  - SJF
  - SRTF
  - Round Robin
  - Priority Scheduling
- Trade-offs exist between:
  - Fairness
  - Efficiency
  - Response time

5.5

# Multi-Processor Scheduling

# Multiprocessor Scheduling

- Multiprocessor $\rightarrow$ Multiple cores $\rightarrow$ Complex scheduling
- Single-core solutions are not sufficient
- No universally best solution

**Type of Cores**

1. Homogeneous: All cores are equally capable
2. Heterogeneous: Some can be faster.

Throughout we mainly assume homogeneous processors

# Approaches to Multiprocessor Scheduling

**Asymmetric Multiprocessing**

- One processor does scheduling
- Others execute tasks
- Master–Slave model
- Not efficient

**Symmetric Multiprocessing (SMP)**

- Each core schedules independently
- Used in practice

# Ready Queue Organization

- Two approaches:
  - Type-I: Single ready queue
  - Type-II: Separate queue per core
- Type-I:
  - All cores share one queue
  - Race condition possible
  - Same process may be selected
  - Locking required
  - Performance degrades
- Type-II:
  - Each core has its own queue
  - Eliminates locking
  - Used in SMP
  - Needs load balancing

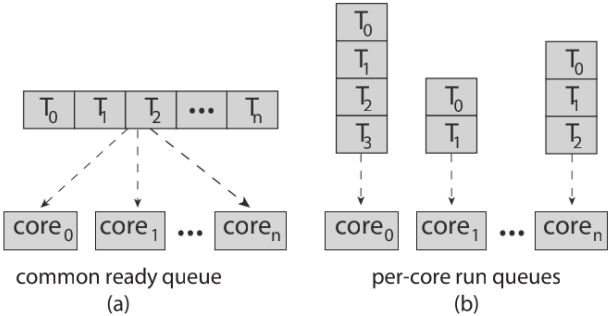


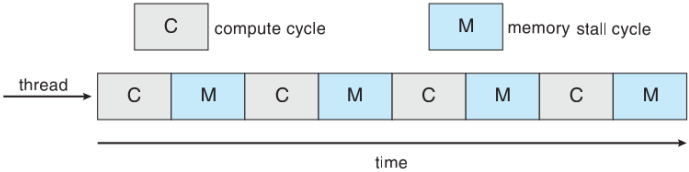

Figure: Organization of ready queues

# Multi-core Processors

- Present day cores are not physically separated
- They are on the same chip
- This creates some difficulties

**Memory Stall can happen if:**

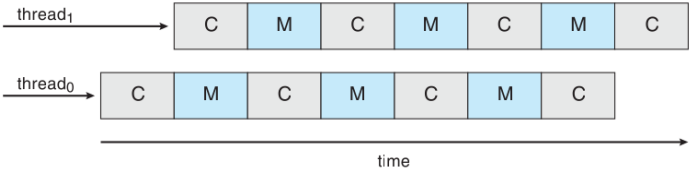
- Memory not in cache
- Data not yet available
- Processor can spend 50% time waiting for data



# Compute Cycle & Memory Stall

**Solution:**

- Two / Multiple hardware threads per core
- Core can switch between threads
- Considers Compute Cycle & Memory Stall Cycle
- Called Dual Threading (Hyperthreading)




**Chip Multithreading (CMT)**

- Can handle many threads per core
- Also called Hyperthreading

# Chip Multithreading

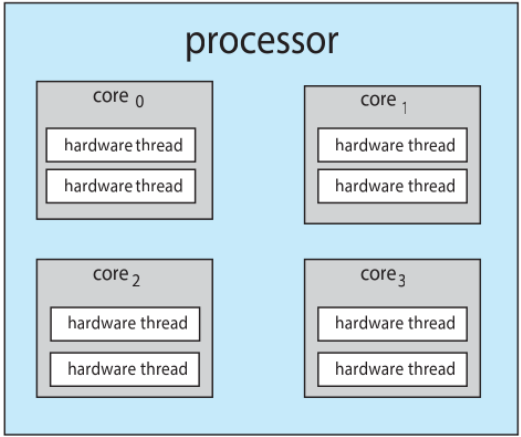



Figure: Physical View

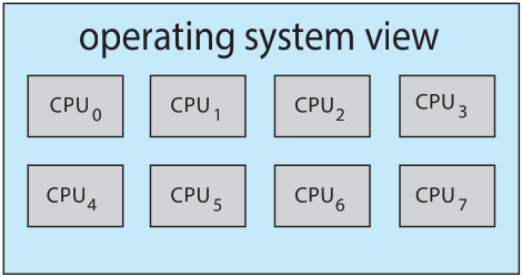


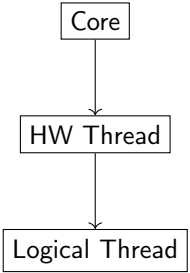

Figure: Logical View

# Levels of Scheduling

- Multi-threaded multicore processor
- Requires two levels of scheduling
- Because a core can run only one physical thread at a time
- It must decide time to switch thread

**Thus two tasks:**

- Choosing physical (hardware) thread
- Choosing logical process/thread from ready queue

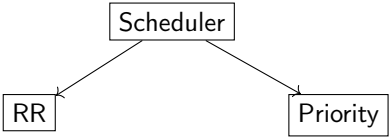


# Scheduling Strategy

- Choosing logical process/thread:
  - Any previous scheduling algorithm can be used

**Hardware thread selection requires strategy:**

- Round Robin can be considered
- Urgency (Priority) can be considered



# Two Levels Scheduling

- Two levels of scheduling are not independent
- If multiple logical threads run on same core, then it is not efficient
- Will not be parallel
- Awareness improves scheduling efficiency

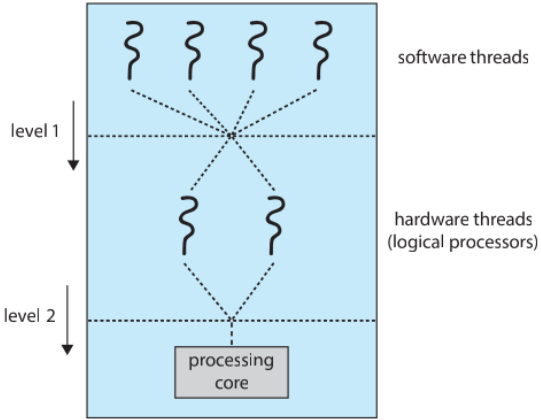



Figure: Two levels of Scheduling

# Load Balancing

- Load balancing is important to distribute workload well
- Required when each core has private ready queue

**Two mechanisms:**

- Push Migration
- Pull Migration

**Push Migration:**

- Kernel checks overloaded processor
- Moves process to another core

**Pull Migration:**

- Idle core pulls process from busy core
- Both may not be mutually exclusive

# Slide 12: Processor Affinity

## What happens to cache when process switches core?

**Problem in thread migration:**

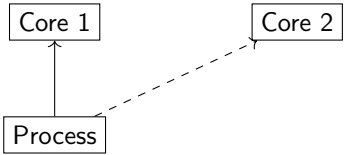
- Cache content invalidated
- New cache must be loaded
- This is costly

**Solution:**

- Avoid migration
- Use warm cache
- Called Processor Affinity

**Types:**

- Hard Affinity:
- Soft Affinity:



# Non-Uniform Memory Access

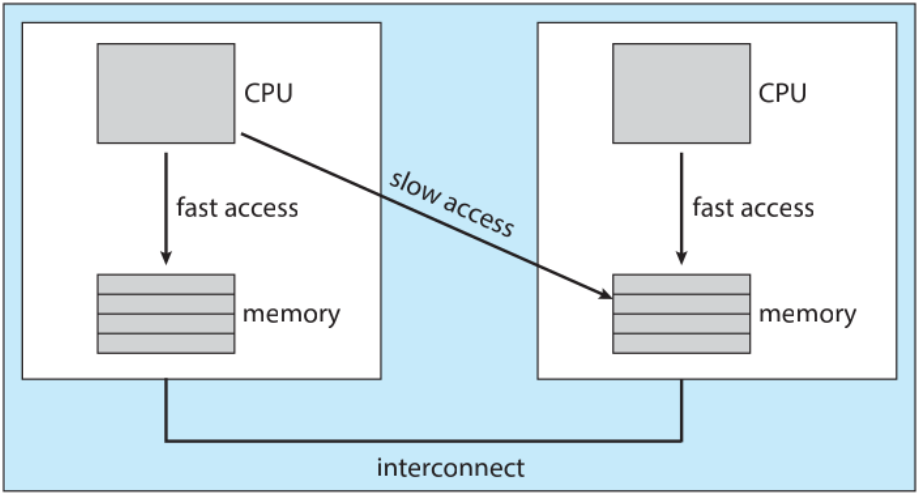
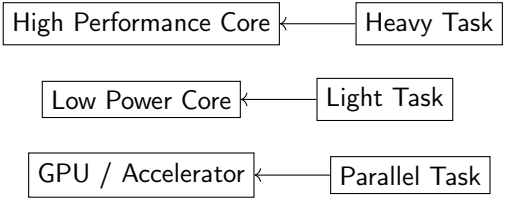



Figure: NUMA and CPU scheduling

# Heterogeneous Multiprocessor Scheduling

- Heterogeneous: Processors (cores) are not identical
- Different cores have different capabilities:
  - Speed / frequency
  - Power consumption
  - Specialized functionality (GPU, AI cores)
- Scheduling becomes more complex
- Tasks must be assigned based on:
  - Performance requirements
  - Energy efficiency
  - Type of workload
- Examples:
  - Big.LITTLE architecture
  - CPU + GPU systems
- Challenges:
  - Load balancing is harder
  - Task-core matching required
  - Possible underutilization of some cores



# References

Textbook:

1. Avi Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Global Edition. Wiley, 2023. ISBN: 9781119320913.

Slides:

1. Avi Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts, Tenth Edition: Lecture Slides*. Wiley, 2018. Available at: `https://os-book.com/OS10/slide-dir/index.html`. Accessed: 2026-03-06.

**Dr. Laltu Sardar**, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/corgos_2026.html.